
**Sunbeam Institute of Information Technology**
**Pune and Karad**

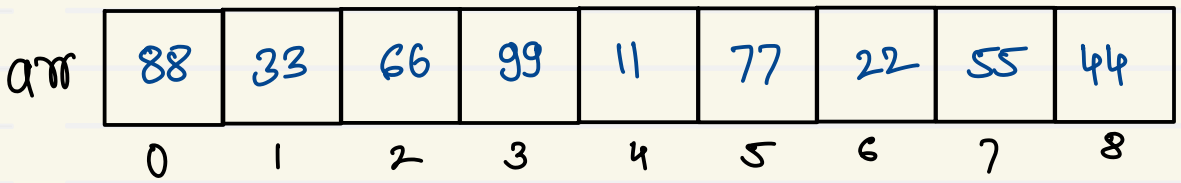
# Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

# Linear search analysis

```
int linearSearch(arr, key){
    for(i=0; i<arr.length; i++){
        if(key == arr[i])
            return i;
    return -1;
}
```




Best case: key = 88 → No. of comps ≈ 1 → O(1)

Avg case: key = 11 → No. of comps ≈ n/2 → O(n)

Worst case: key = 44/89 → No. of comps ≈ n → O(n)

No. of comparisons = n

Time ∝ No. of comparisons

Time ∝ n

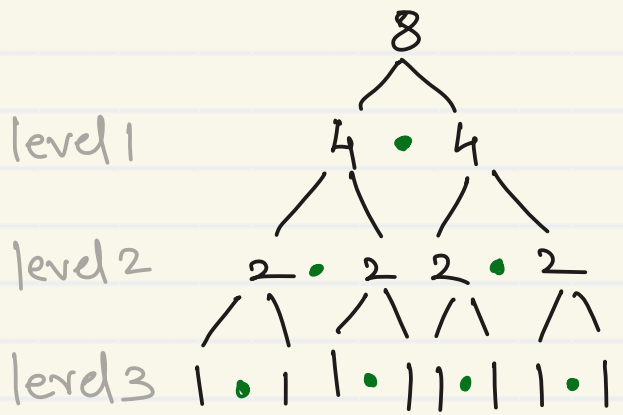
$T(n) = O(n)$

Processing variables = key & i

Auxilliary space → constant

$S(n) = O(1)$

# Binary search analysis




$2^3 = 8$

per level one comparison will done

l - no. of level

n - no. of element

$$2^l = n$$

processing variable - key, left, right, mid

Auxilliary space → constant

$$S(n) = O(1)$$

$$2^l = n$$

$$\log 2^l = \log n$$

$$l \log 2 = \log n$$

$$l = \frac{\log n}{\log 2}$$

$$comps = \frac{\log n}{\log 2}$$

$$Time \propto \frac{1}{\log 2} \log n$$

$$T(n) = O(\log n)$$ ← Avg, ← Worst

if key is found in first comparison

$$T(n) = O(1)$$ ← Best

# Searching algorithms analysis

- Time is directly proportional to number of comparisons
- For searching and sorting algorithms, count number of comparisons done

1. Linear search
   - Best case - if key is found at few initial locations → O(1)
   - Average case - if key is found at middle locations → O(n)
   - Worst case - if key is found at last few locations / key is not found → O(n)

S(n)=O(1)

2. Binary search
   - Best case - if key is found at first few levels → O(1)
   - Average case - if key is found at middle levels → O(log n)
   - Worst case - if key is found at last level / not found → O(log n)

S(n)=O(1)

- function calling itself is called as recursion

- to do recursion we must know,
  1. formula/process in terms of itself
  2. terminating / base condition.

e.g. $n! = n * (n-1)!$ $\quad 0! = 1! = 1$

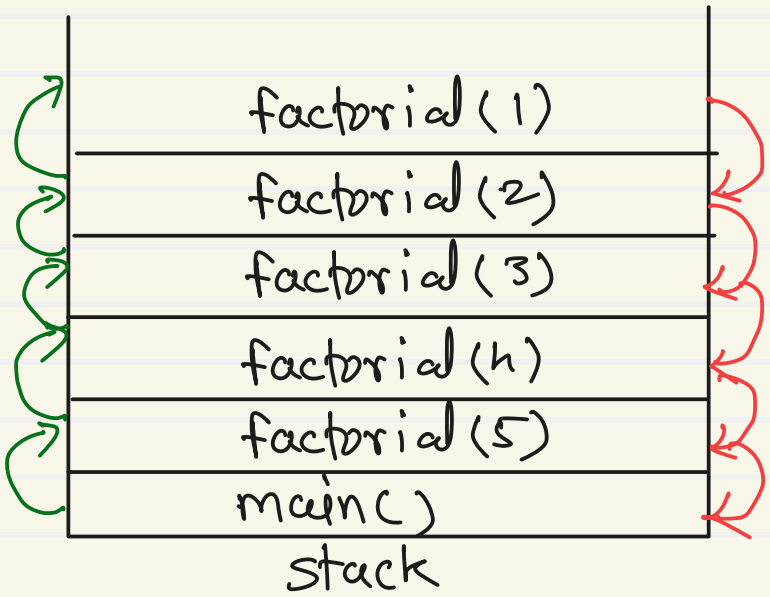
$b^i = b * b^{i-1}$ $\quad b^0 = 1$

$T_n = T_{n-1} + T_{n-2}$ $\quad T_1 = T_2 = 1$

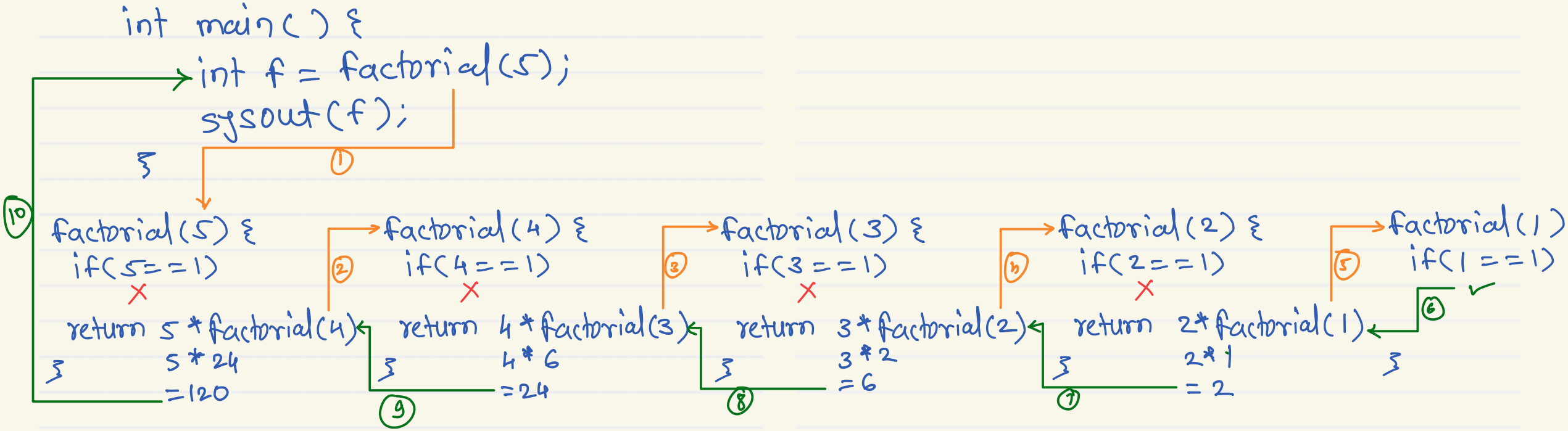
e.g. 1, 1, 2, 3, 5, 8, 13 ....

- for every function call, seperate stack frame (Function Activation Record) is created on process stack.

```
int factorial(int n){
    // check base condition
    if(n==1)
        return 1;
    //calcalate factorial
    return n * factorial(n-1)
}
```



# Factorial using recursion



# Algorithm implementation approaches

## Iterative approach

```
int factorial(int n) {
    int fact = 1;
    for(i=1; i<=n; i++)
        fact *= i;
    return fact;
}
```

Time ∝ No. of iterations of the loop

Time ∝ n

$$T(n) = O(n)$$

$$S(n) = O(1)$$

## Recursive approach

```
int factorial(int n) {
    if(n==0)
        return 1;
    return n * factorial(n-1);
}
```

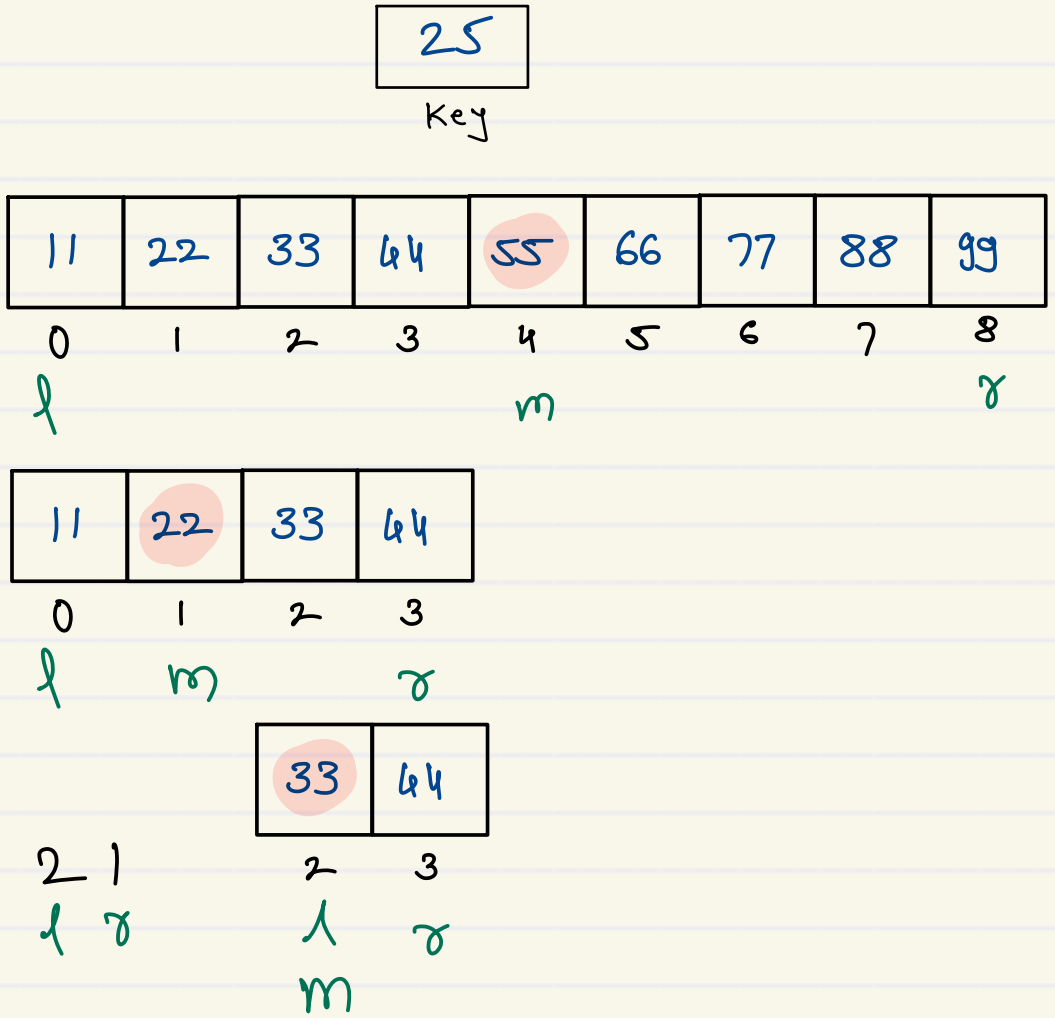
Time ∝ no. of recursive calls

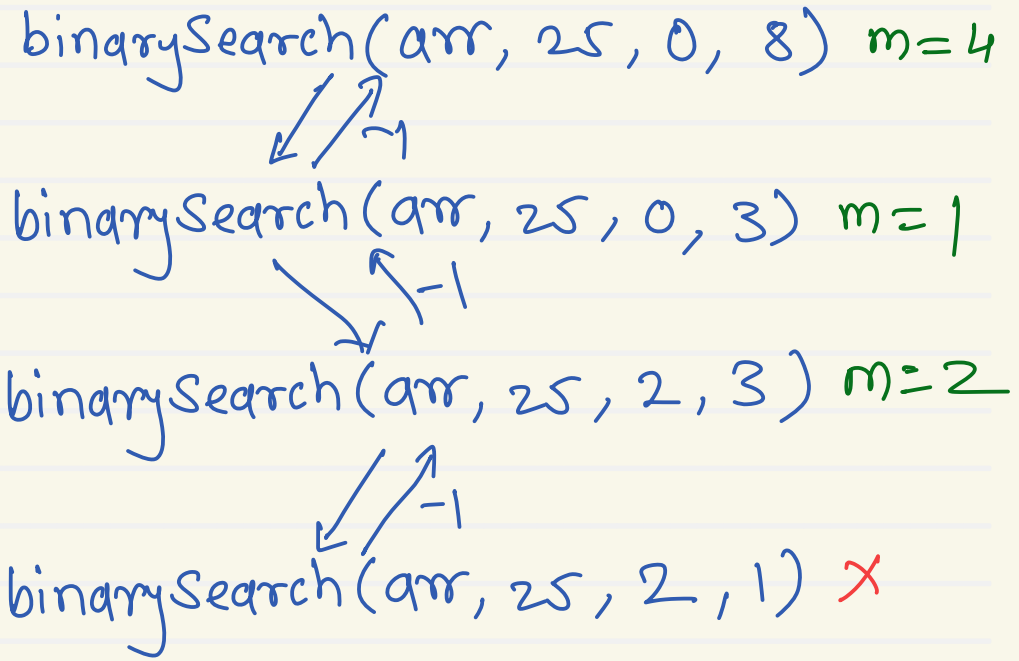
Time ∝ n

$$T(n) = O(n)$$

$$S(n) = O(n)$$ ← space required to create FARs

# Binary Search using Recursion

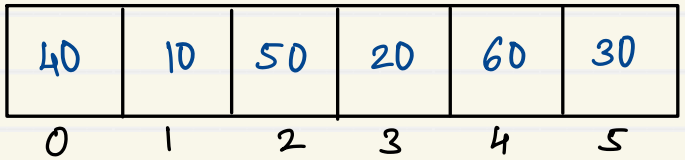


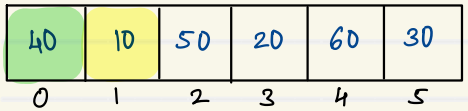

$$T(n) = O(\log n)$$
$$S(n) = O(\log n)$$

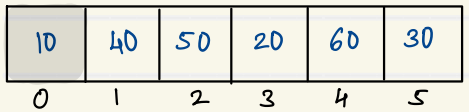
# Selection sort

1. Select one position of the array
2. Find smallest element out of remaining elements
3. Swap selected position element and smallest element
4. Repeat above steps until array is sorted (N-1)

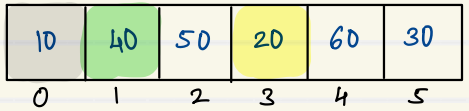


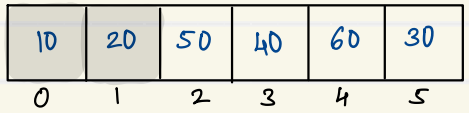

Pass 1



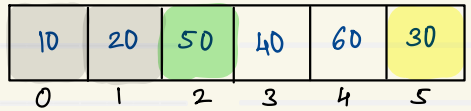



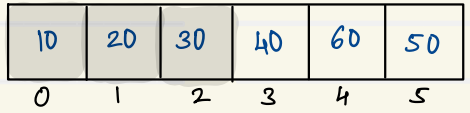

Pass 2



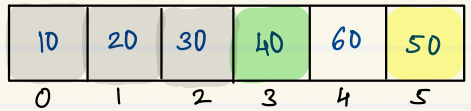



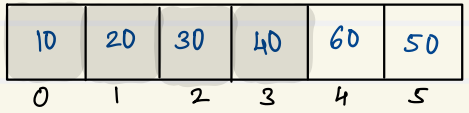

Pass 3



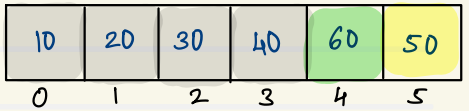



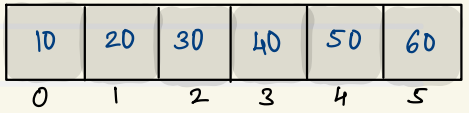

Pass 4




Pass 5



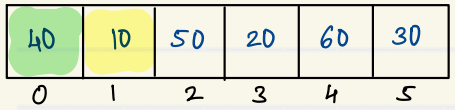

to select positions : $i = 0$ to $N-2$ $(i < N-1)$
to find smallest element : $j = i+1$ to $N-1$ $(j < N)$

# Selection sort

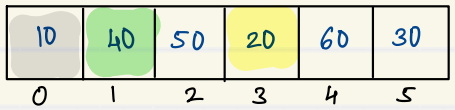
```
int minIndex = i;
for(j=i+1; j<N; j++)
    if(arr[j] < arr[minIndex])
        minIndex = j;
```

| Passes | Comps |
|---|---|
| 1 | n-1 |
| 2 | n-2 |
| 3 | n-3 |
| ⋮ | ⋮ |
| n-2 | 2 |
| n-1 | 1 |

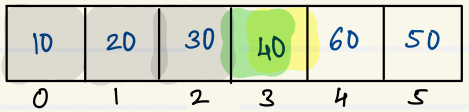
n → no. of element
n-1 → no. of passes




| i | minIndex | j |
|---|---|---|
| 0 | 0 | 1 |
| | 1 | 2 |
| | | 3 |
| | | 4 |
| | | 5 |
| | | 6 X |




| i | minIndex | j |
|---|---|---|
| 1 | 1 | 2 |
| | 3 | 3 |
| | | 4 |
| | | 5 |
| | | 6 X |




| i | minIndex | j |
|---|---|---|
| 3 | 3 | 4 |
| | | 5 |
| | | 6 X |

Total comps = $1+2+3+\dots+(n-2)+(n-1)$

$= \frac{n(n-1)}{2}$

Time $\propto \frac{1}{2}(n^2-n)$

$T(n) = O(n^2)$ worst Avg best

Processing variable : i, j, N, minIndex, temp

Auxiliary space → constant

$S(n) = O(1)$

Mathematical polynomials:
functions

Degree: Highest power of a variable

- while writing time / space complexities consider only degree term

e.g. $Time \propto \frac{1}{2}(n^2 - n)$

$T(n) = O(n^2)$

- because degree term is highest growing term in polynomial

| $n$ | $n^2$ |
|---|---|
| 1 | 1 |
| 10 | 100 |
| 100 | 10000 |
| 1000 | 1000000 |

# Bubble sort

1. Compare all pairs of consecutive elements of the array one by one
2. If left element is greater than right element , then swap both
3. Repeat above steps until array is sorted (N-1)

n - no. of elements

| passes | comps |
|---|---|
| 1 | n-1 |
| 2 | n-2 |
| 3 | n-3 |
| ⋮ | ⋮ |
| | 2 |
| n-1 | 1 |

Total comps = $1+2+3+\ldots\ldots(n-2)+(n-1)$ (n)

Total comps = $\frac{n(n+1)}{2}$

$= \frac{n^2+n}{2}$

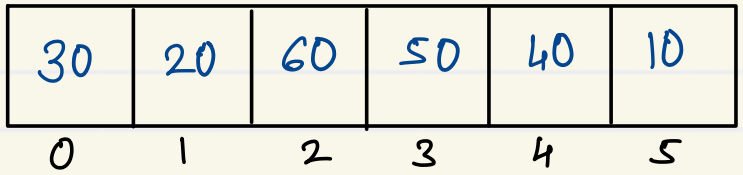
Time $\propto$ comps

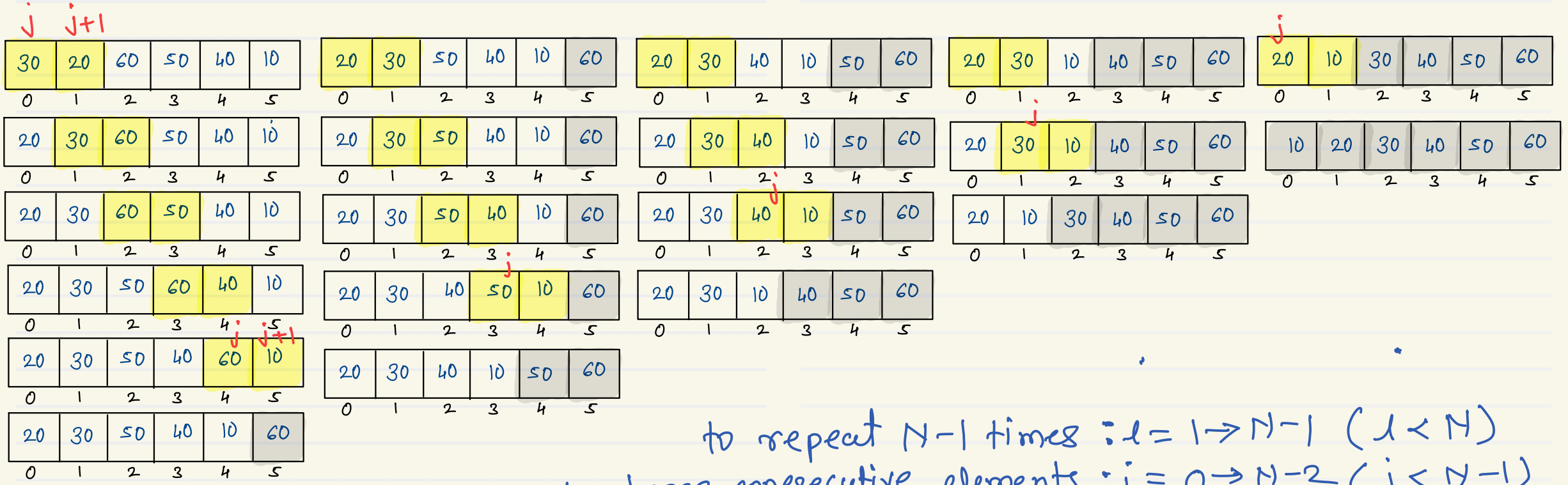
Time $\propto \frac{1}{2}(n^2+n)$

$T(n) = O(n^2)$ Avg Worst

$S(n) = O(1)$ ← in place sorting algo.

# Bubble sort



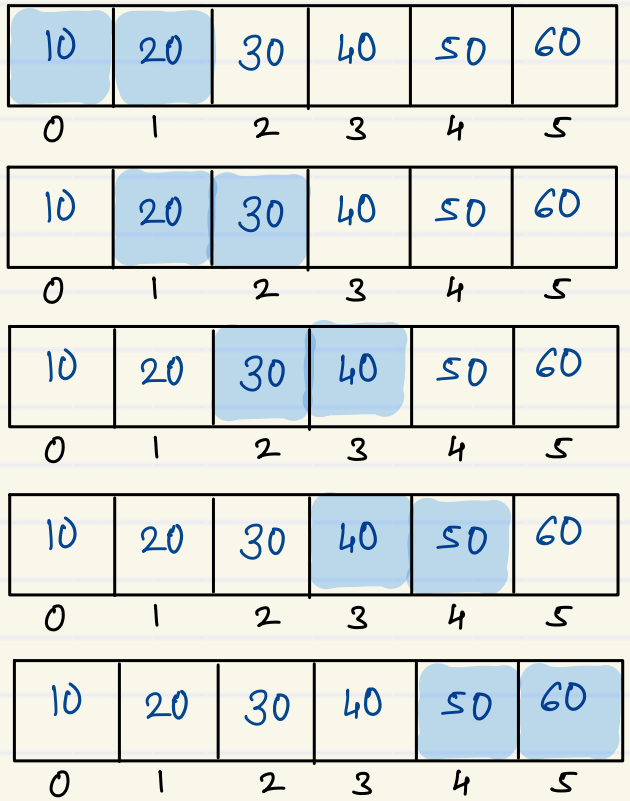

to repeat N-1 times : $i = 1 \rightarrow N-1$ $(i < N)$

to choose consecutive elements : $j = 0 \rightarrow N-2$ $(j < N-1)$

$i = 1$ to $N-1$ $(i < N)$

| $i$ | $j < N-1$ | $j < N-i$ | |
|---|---|---|---|
| 1 | $0 \to 4$ | $0 \to 4$ | $< 5$ |
| 2 | $0 \to 4$ | $0 \to 3$ | $< 4$ |
| 3 | $0 \to 4$ | $0 \to 2$ | $< 3$ |
| 4 | $0 \to 4$ | $0 \to 1$ | $< 2$ |
| 5 | $0 \to 4$ | $0 \to 0$ | $< 1$ |

# Bubble sort




```
for(i=1; i<N; i++) {
    flag=0
    for(j=0; j<N-i; j++ {
        if(arr[j] > arr[j+1]) {
            SWAP(arr[j], arr[j+1];
            flag=1
        }
    }
    if(flag==0)
        break;
}
```

n - no. of elements

No. of comps = n-1

Time $\propto$ n-1

$T(n) = O(n)$ Best